

Mobile Application Development with Flutter

Lecture 46. State Management in Flutter

What is State?

- State is any data that can change during the lifecycle of a widget and affects the UI
- Flutter is a **declarative framework**, we don't "change" the UI directly
- instead, we update the state, and Flutter automatically rebuilds the UI to reflect those changes



Examples

- Counter value
- Selected dropdown item
- Switch On/Off
- User input (Text Field)
- API response

What is State Management?

- State management is the way we handle, update, and share changing data in an app so the UI updates correctly

 Simple Meaning

 “How our app remembers and updates data”

Prepared by Ehtisham Rashed

Types of State

- ◆ Local State (Ephemeral)

- Used inside one widget
- Example:
 - Toggle switch
 - Button click

- ◆ Global State (App)

- Shared across multiple screens
- Example:
 - User login
 - Cart item
 - Theme

Basic State Management (setState)

- setState is the simplest form of state management

- How it works

- Data changes
- setState triggers rebuild
- UI update

```
int counter = 0;

ElevatedButton(
  onPressed: () {
    setState(() {
      counter++;
    });
  },
  child: Text("$counter"),
)
```

Basic State Management (setState)

⚠ Problem with setState

- Works only in one widget
- Difficult to manage shared data
- Unnecessary rebuild
- Hard to manage big apps

Advanced State Management (Industry Level)

◆ Popular Methods

- Provider
- Riverpod
- Bloc
- GetX

🧠 Concept Behind All

- Keep state separate from UI and update it in a structured way

Simple Analogy



Example: Classroom

- Teacher = State Manager
- Students = UI
- Board = Data



When teacher updates board → all students see change

When to Use What?

● Use setState when:

- Small apps
- Single screen
- Simple UI

● Use Provider / Riverpod when:

- Medium apps
- Shared data

● Use Bloc / GetX when:

- Large apps
- Complex logic